

10 Questions — Duck Typing 🦆

Q1 — Easy

Create **3 classes** Dog, Cat, Bird

- Each has a method `speak()`
 - Dog → Woof!, Cat → Meow!, Bird → Tweet!
 - Create a function `make_sound(animal)`
 - Call it with all 3 objects in a **loop**
-

Q2 — Easy

Create **3 classes** Laptop, Mobile, Tablet

- Each has a method `turn_on()` and `turn_off()`
 - Laptop → Windows starting!, Mobile → Android starting!
 - Tablet → iPad starting!
 - Create function `start_device(device)`
 - Call `turn_on()` and `turn_off()` for all devices
-

Q3 — Easy

Create **3 classes** Chef, Doctor, Teacher

- Each has a method `work()` and `introduce()`
 - Chef → cooking food, Doctor → treating patients
 - Teacher → teaching students
 - Create function `do_work(person)`
 - Loop through all 3 objects and call both methods
-

Q4 — Medium

Create **4 classes** YouTube, Netflix, Spotify, Podcast

- Each has methods `play(title)` and `pause()` and `stop()`
- Create function `stream(platform, title)`
- Each platform shows its OWN message when playing

- Loop and test **all platforms** with different titles
-

Q5 — Medium

Create **4 classes** PDF, Word, Excel, PowerPoint

- Each has methods:
 - open(filename) → shows opening message
 - save(filename) → shows saving message
 - close() → shows closing message
 - Create function handle_file(app, filename)
 - Test all 4 file types
-

Q6 — Medium

Create **4 classes** for delivery:

BikeDelivery, CarDelivery, DroneDelivery, BoatDelivery

- Each has methods:
 - pickup(item) → picks up item
 - deliver(item, address) → delivers to address
 - confirm() → confirms delivery
 - Create function process_delivery(vehicle, item, address)
 - Test all 4 delivery types
-

Q7 — Medium

Create **4 classes** for payments:

CreditCard, DebitCard, UPI, NetBanking

- Each has:
 - private __balance
 - method validate(amount) → checks if balance enough
 - method pay(amount) → deducts and shows message
 - method receipt() → shows receipt
- Create function make_payment(method, amount)

- Test valid and invalid amounts for each
-

Q8 — Hard

Create **4 classes** for notifications:

EmailAlert, SMSAlert, WhatsAppAlert, PushAlert

- Each has:
 - private __recipient
 - private __message_count = 0
 - method send(message) → sends and increases count
 - method get_count() → returns message count
 - static method is_valid_message(msg) → msg not empty
 - method show_stats() → shows recipient + count
 - Create function alert_user(alert, message)
 - Send **3 messages** to each and show stats
-

Q9 — Hard

Create **5 classes** for a restaurant:

Pizza, Burger, Sushi, Pasta, Salad

- Each has:
 - private __name, __price
 - method prepare() → shows preparation steps
 - method serve() → shows serving message
 - method bill() → shows price
 - static method is_valid_price(price) → price > 0
 - class attribute total_orders = 0
 - class method get_orders() → shows total orders
- Create function process_order(food)
→ calls prepare, serve, bill in order
- Order **2 of each** food item
- Show **total orders** at end

Q10 — All Together

Create **5 classes** for a bank:

SavingsAccount, CurrentAccount,
LoanAccount, FixedDeposit, CryptoWallet

- Each has:
 - private `__owner`, `__balance`
 - accessors + mutators with validation
 - method `deposit(amount)` → validates then adds
 - method `withdraw(amount)` → validates then deducts
 - method `calculate_interest()` → different for each:
 - Savings → 5%, Current → 2%,
 - Loan → 10%, Fixed → 8%, Crypto → 15%
 - method `show()` → prints all details + interest
 - static method `is_valid_amount(amount)` → `amount > 0`
 - class attribute `total_accounts = 0`
 - class method `get_total()` → shows total accounts

Create function `process_account(account, deposit_amt, withdraw_amt)`
→ deposits, withdraws, shows details

Test:

- Create **one object of each** account type
- Process each account
- Show **total accounts** at end
- Test **invalid amounts** for each